ADS205 Tutorial 2 - Deterministic and Nondeterministic Finite Automata

September 9, 2026

1 ADS205 — Decision, Computation and Language

2 Tutorial 2 — Deterministic and Nondeterministic Finite Automata

Solutions are folded beneath each problem — click the arrow to expand them. Attempt the problem first.

2.1 Reference card

Object	Definition
DFA	A tuple $M = (Q, \Sigma, \delta, q_0, F)$ with $\delta : Q \times \Sigma \to Q$
NFA	A tuple $N = (Q, \Sigma, \delta, q_0, F)$ with $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q)$
Accepting a string (DFA)	The unique computation ends in a state in F after all input is read
Accepting a string (NFA)	At least one computation path consumes all input and ends in a state in F
ε-transition	Changes state without consuming an input symbol
Regular language	A language recognized by some DFA, equivalently by some NFA

Two rules worth keeping visible:

1. A machine stays in a state only when an explicit self-loop permits it.
2. A dead NFA branch does not reject the whole string if another branch accepts.

---

3 Part A — Concepts and computation paths

3.1 Problem 1 — Quick diagnostic

Answer true or false and justify briefly.

1. A DFA must have exactly one accepting state.
2. A DFA has exactly one computation path for each input string.
3. If an NFA has no transition from the current state on the next symbol, that computation stays in the same state.
4. An NFA accepts only when every computation path accepts.
5. Two different DFAs can recognize the same language.
6. An ε-transition consumes the empty string, so the input position does not change.

Solution — click to expand

1. **False.** The set F may contain zero, one, or several states.
2. **True.** For every state and input symbol, the DFA transition function chooses exactly one next state.
3. **False.** That path dies. Staying requires an explicit self-loop on that symbol.
4. **False.** One accepting path is sufficient.
5. **True.** A DFA may contain redundant or unreachable states and still recognize the same language as another DFA.
6. **True.** It changes the state without consuming input.

---

4 Part B — Constructing finite automata

4.1 Problem 2 — A DFA for strings ending in `ab`

Let $\Sigma = \{\texttt{a}, \texttt{b}\}$. Construct a DFA for

$$L = \{w \in \Sigma^* \mid w \text{ ends with } \texttt{ab}\}.$$

Give the meaning of every state and a complete transition table. Test your machine on ε, `ab`, `aab`, `aba`, and `bbab`.

Solution — click to expand

Use three states:

- q_0: the input read so far does not end in `a` or `ab`;
- q_1: it ends in `a` but not `ab`;
- q_2: it ends in `ab`.

The start state is q_0 and $F = \{q_2\}$.

State	`a`	`b`
q_0	q_1	q_0
q_1	q_1	q_2

State	a	b
q_2	q_1	q_0

The five test results are reject, accept, accept, reject, accept.

4.2 Problem 3 — An NFA for strings containing 01

Let $\Sigma = \{0, 1\}$. Construct an NFA for

$$L = \{u01v \mid u, v \in \Sigma^*\}.$$

Explain where nondeterminism occurs and why one successful branch is enough.

Solution — click to expand

State	0	1
q_0	$\{q_0, q_1\}$	q_o
q_1	$\emptyset$	q_2
q_2	q_2	q_2

with start state q_0 and accepting state q_2.

At q_0, reading 0 creates two possibilities: remain at q_0 to keep searching, or move to q_1 and guess that this 0 starts the target substring. If the next symbol is 1, that branch reaches q_2. The self-loops on q_2 accept every possible suffix v.

---

5 Part C — From NFA to DFA

5.1 Problem 4 — Subset construction with ε-moves

Let N have states $\{s, p, f\}$, alphabet $\{0, 1\}$, start state s, accepting state f, and transitions

State	ε	0	1
s	$\{p\}$	$\{s\}$	$\emptyset$
p	$\emptyset$	$\emptyset$	$\{f\}$
f	$\emptyset$	$\{f\}$	$\{f\}$

1. Compute ε-closure($\{s\}$), where ε-closure() indicates the set of states that a state can reach with ε-transitions.
2. Apply the subset construction to show the DFA states.
3. Which DFA state-sets are accepting?
4. Describe the recognized language.

Solution — click to expand

1. ε-closure$(\{s\}) = \{s, p\}$, the DFA start state.
2. The reachable DFA states and transitions are

DFA state	`0`	`1`
$\{s, p\}$	$\{s, p\}$	$\{f\}$
$\{f\}$	$\{f\}$	$\{f\}$

3. A state-set is accepting exactly when it contains f, so $\{f\}$ is accepting.
4. The language consists of binary strings containing at least one `1`.

5.2 Problem 5 — Closure under intersection and complement

1. Given DFAs for regular languages A and B, construct a DFA for $A \cap B$.
2. Given a **complete** DFA for A, construct a DFA for $\overline{A}$.
3. Why is flipping final states in an arbitrary NFA incorrect?

Solution — click to expand

For intersection, use product states $Q_A \times Q_B$, start state (s_A, s_B), transition

$$\delta((q, r), a) = (\delta_A(q, a), \delta_B(r, a)),$$

and accepting set $F_A \times F_B$. After every prefix, the pair records exactly the two component states.

For complement, keep a complete DFA unchanged except for replacing F by $Q \setminus F$. Every input has one complete computation, so acceptance is reversed.

An NFA accepts when at least one path accepts. Flipping final states asks whether at least one formerly rejecting path exists, which is different from asking whether no formerly accepting path exists. Determinize and complete the NFA first.

5.3 Problem 6 — Two operations on regular languages

For $w = w_1 \cdots w_n$, define

$$\text{STRETCH}(w) = w_1 w_1 w_2 w_2 \cdots w_n w_n.$$

For $|w| \geq 2$, define CHOP(w) by deleting the first and last symbols. Prove:

1. if L is regular, then STRETCH(L) is regular;
2. if L is regular, then CHOP(L) is regular.

Solution — click to expand

Let M be a DFA for L.

For STRETCH, replace every transition $p \xrightarrow{a} q$ by a two-step path

$$p \xrightarrow{a} r_{p,a} \xrightarrow{a} q,$$

using a fresh intermediate state for that transition. The resulting NFA accepts exactly the strings obtained by doubling every symbol of a string accepted by M.

For CHOP, build an NFA that first guesses the deleted initial symbol a, enters the state $\delta(s, a)$ without consuming the chopped input, then simulates M on the middle string. At the end it accepts from a simulated state q exactly when some final symbol b satisfies $\delta(q, b) \in F$. Equivalently, use a fresh start state with ε-moves to every $\delta(s, a)$ and declare q accepting when it can reach F in one symbol. This recognizes precisely the middle portions of accepted strings of length at least two.

---

6 Part E — Optional Python simulation

The following code uses transition tables directly. DFA states are single values; NFA configurations are sets of possible states. No external package is required.

[1]:
```python
from collections import deque


def run_dfa(transitions, start, accepting, word):
    """Return the state trace and whether a DFA accepts word."""
    state = start
    trace = [state]
    for symbol in word:
        state = transitions[state][symbol]
        trace.append(state)
    return trace, state in accepting


def epsilon_closure(states, transitions):
    """All states reachable using zero or more epsilon transitions."""
    closure = set(states)
    agenda = list(states)
    while agenda:
        state = agenda.pop()
        for nxt in transitions.get(state, {}).get("", set()):
            if nxt not in closure:
                closure.add(nxt)
                agenda.append(nxt)
    return closure


def run_nfa(transitions, start, accepting, word):
    """Return reachable state-sets and whether an epsilon-NFA accepts word."""
    current = epsilon_closure({start}, transitions)
    trace = [frozenset(current)]
    for symbol in word:
        moved = set()
```

```
        for state in current:
            moved |= transitions.get(state, {}).get(symbol, set())
        current = epsilon_closure(moved, transitions)
        trace.append(frozenset(current))
    return trace, bool(current & accepting)


def determinize(transitions, start, accepting, alphabet):
    """Build the reachable part of an equivalent DFA."""
    dfa_start = frozenset(epsilon_closure({start}, transitions))
    agenda = deque([dfa_start])
    seen = {dfa_start}
    dfa = {}
    while agenda:
        subset = agenda.popleft()
        dfa[subset] = {}
        for symbol in alphabet:
            moved = set()
            for state in subset:
                moved |= transitions.get(state, {}).get(symbol, set())
            target = frozenset(epsilon_closure(moved, transitions))
            dfa[subset][symbol] = target
            if target not in seen:
                seen.add(target)
                agenda.append(target)
    dfa_accepting = {subset for subset in seen if subset & accepting}
    return dfa, dfa_start, dfa_accepting
```

[ ]:
```
# Check the DFA from Problems 2.
ends_ab = {
    "q0": {"a": "q1", "b": "q0"},
    "q1": {"a": "q1", "b": "q2"},
    "q2": {"a": "q1", "b": "q0"},
}

for word in ["", "ab", "aab", "aba", "bbab"]:
    trace, accepted = run_dfa(ends_ab, "q0", {"q2"}, word)
    print(f"{word!r:6}  {trace}  {'accept' if accepted else 'reject'}")
```

```
''      ['q0']  reject
'ab'    ['q0', 'q1', 'q2']  accept
'aab'   ['q0', 'q1', 'q1', 'q2']  accept
'aba'   ['q0', 'q1', 'q2', 'q1']  reject
'bbab'  ['q0', 'q0', 'q0', 'q1', 'q2']  accept
```

[ ]:
```
# Check the NFA from Problems 3.
contains_01 = {
    "q0": {"0": {"q0", "q1"}, "1": {"q0"}},
```

```
    "q1": {"1": {"q2"}},
    "q2": {"0": {"q2"}, "1": {"q2"}},
}

trace, accepted = run_nfa(contains_01, "q0", {"q2"}, "10010")
print("trace:", [set(states) for states in trace])
print("accepted:", accepted)
```

```
trace: [{'q0'}, {'q0'}, {'q0', 'q1'}, {'q0', 'q1'}, {'q2', 'q0'}, {'q2', 'q0',
'q1'}]
accepted: True
```

6.1 What to take away

- A DFA follows one path; an NFA tracks a set of possible states and accepts when at least one path succeeds.
- Missing transitions kill an NFA branch. They never mean "stay where you are."
- Subset construction turns each DFA state into a set of NFA states; ε-closure belongs at the start and after every symbol move.
- Product states let one DFA remember two facts at once, which proves closure under intersection.
- Complementing works by flipping final states of a **complete DFA**. Determinize an NFA first.